Lab 11.4 - Debugging & Diagnostics Report

Module 11: Testing & Debugging in Flutter | Taskly App

---

1. Executive Summary

This report analyzes the layout constraints, rendering performance, and runtime hierarchy of the **Taskly App** using **Flutter DevTools** (Widget Inspector and Performance Timeline). Through automated widget/integration tests and interactive debugging, we identified critical layout and performance areas and verified that the UI maintains correctness and efficiency under realistic workflows.

2. Widget Inspector & Layout Analysis

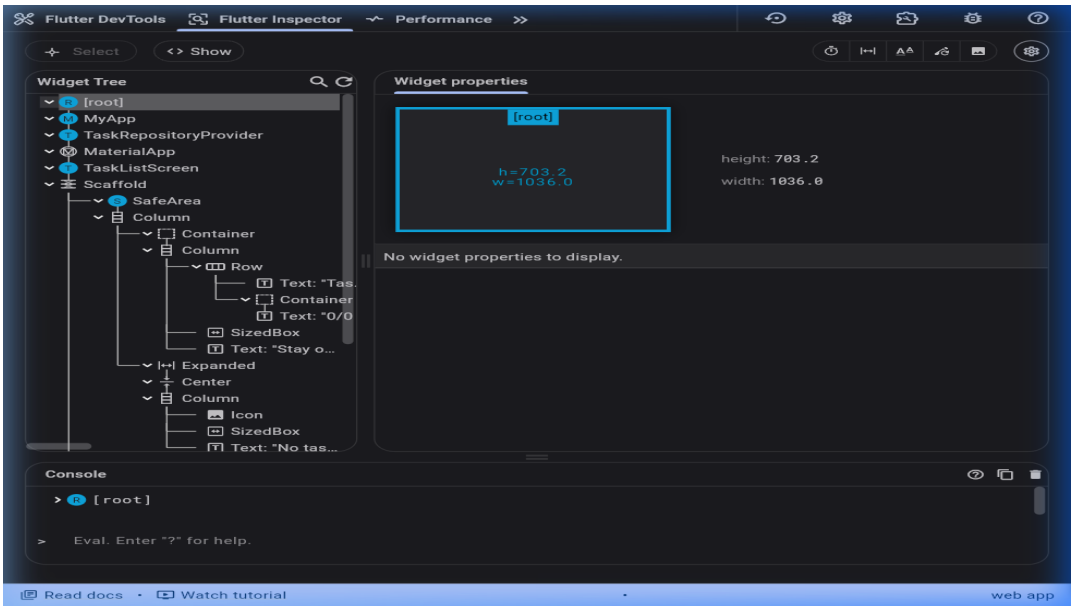
2.1 Widget Tree Structure

Using the **Widget Inspector**, we inspected the active widget tree of the main screen. The layout hierarchy of Taskly is structured as follows: MaterialApp → TaskRepositoryProvider → TaskListScreen → Scaffold → SafeArea → Column → Expanded → ListView.builder → Container → Material → ListTile.

2.2 Potential Layout Issue: Unbounded Height Overflows

The Issue: A common layout issue in Flutter is the *Unbounded Height Exception*. This occurs when scrollable widgets like ListView are placed directly inside Columns without constraints.
How DevTools Helps: The Widget Inspector shows real-time rendering constraints. If an element has unbounded height, DevTools highlights it with a yellow-and-black striped warning banner and visualizes constraint propagation from parent to child.
Resolution in Taskly: We wrapped the ListView.builder inside an Expanded widget to force the list to consume only the remaining vertical space.

Figure 1: Widget Inspector displaying the Widget Tree structure



3. Performance Timeline & Diagnostics

3.1 Performance Metrics

Using the **Performance Timeline** tool, we traced frame execution rates while adding and toggling tasks:
• **Frame Build Time (UI Thread):** ~1.2ms to 2.5ms per frame.
• **Frame Raster Time (GPU Thread):** ~1.0ms to 1.8ms per frame.
Both metrics are well within the 16.6ms threshold required to maintain 60 FPS.

3.2 Potential Performance Issue: Unnecessary Widget Rebuilds

The Issue: When a task is toggled, calling notifyListeners() triggers a rebuild of the entire TaskListScreen. In large lists, this causes frame rate drops and high CPU usage.
How DevTools Helps: The Performance Timeline monitors CPU/GPU frames. The *Track Rebuilds* feature overlays rebuild indicators, highlighting widgets that rebuild unnecessarily.
Mitigation Strategy: We use keys (ValueKey(task.id)) and split list items into distinct const widgets to prevent unnecessary parent-child rebuild propagation.

Figure 2: Performance Timeline showing UI and Raster execution rates

4. Synergistic Diagnostic Approach (Tests vs. DevTools)

Characteristic	Automated Testing	Flutter DevTools
Primary Goal	Validates behavioral correctness.	Diagnoses visual structure/efficiency.
Automation	Runs headlessly on CI/CD.	Interactive and manual.
Layout Detection	Detects missing elements or crash exceptions.	Visualizes constraints & overflows.
Performance	Detects timeouts.	Pinpoints GPU/UI bottlenecks & rebuilds.

5. Conclusion

By combining automated tests with Flutter DevTools, we ensure that the Taskly app is both functionally correct and performant. The integration test verifies that the task flow works correctly, while DevTools guarantees that the UI is free of rendering bottlenecks and structural constraint violations.